



DPURubyScript

This DPU offers the option to implement additional DPUs in the programming language Ruby. (From now, these will be called RPU, "Ruby Programming Units".)

1. Inputs:

All inputs are defined in the Ruby script by the user.

2. Static parameters:

<i>Script</i>	Name of the Ruby script. If the file name isn't absolute, it will be searched within the folders <SILAB>\lib\scripts and in the project folder.
---------------	---

3. Outputs:

All outputs are defined in the Ruby script by the user.

4. Functional principle:

Ruby is a dynamically typed and completely object oriented programming language which offers extensive functionality with a compact and clear syntax.

A large amount of tutorials and manuals are available at its homepage <http://www.ruby-lang.org/> and additionally at <http://www.ruby-doc.org/>. The book "Programming Ruby" will be especially helpful and can be found online under <http://www.ruby-doc.org/docs/ProgrammingRuby/>.

A reference of the standard Ruby classes and modules which are available in all scripts, can be accessed at <http://ruby-doc.org/core-2.3.0/>.

Note: The Ruby interpreter which is integrated in SILAB only contains the *Ruby Core API*, not *the Ruby Std-lib*.

When developing SILAB RPUs, a few specialties should be noted, which are described in the document "Reference Ruby Scripting".

The following items should be noted in addition when developing a RPU:

- The script's main class must be named like the script file (without folder and extension). For example, the script file *D:\SILAB\Projects\Demo\scripts\RPUDemo.rb* must contain the Ruby class *RPUDemo*.
- The script's main class must inherit from the RPU class, which is described more extensively in "Reference Ruby Scripting".



SILAB DRIVING SIMULATION

version Version 7.1 documentation

- RPU inputs and outputs can be created in the initialize method of the RPU class by calling `var_in` and `var_out` (cf. “Reference Ruby Scripting”).
- During the simulation, SILAB calls the following methods of the RPU class when the corresponding event occurs:
 - `prepare()` When loading the configuration
 - `start()` When launching the simulation
 - `trigger(t, te)` Once for each simulation step
 - `stop()` When stopping the simulation
 - `release()` When exiting the simulation(‘Shutdown’ button in SILAB or exiting the SILAB application)

The section ‘Example’ (below) describes how to integrate a Ruby script which satisfies these conditions into the SILAB system just like another DPU. For that purpose, the Ruby script’s file name is passed to an instance of the `DPURubyScript`. This instance then has the inputs and outputs that have been defined as RPU class members using `var_in` and `var_out`.

5. Example:

The following example illustrates how a simple DPU can be written in Ruby and how it can be integrated into the SILAB system. The RPU should have one input and one output. The output computes, in each simulation step, the mean value of the last three input values. The corresponding Ruby program is shown below. Please note the source code comments.

```
# ---- Listing of the Ruby script RPUMeanValue.rb ----
#
# The file RPUMeanValue.rb must contain the class RPUMeanValue,
# which must inherit from the builtin RPU class.
class RPUMeanValue < RPU

  # Statements entered in the initialize method are executed once
  # when the configuration is loaded.
  def initialize()
    var_in :@input # Creates a DPU input named 'Input'
    var_out :@output # Creates a DPU output named 'Output'
  end

  # Statements entered in the start method are executed once when
  # the simulation is started.
  def start()
    # Create two variable to save temporary values from the
```



SILAB DRIVING SIMULATION

version Version 7.1 documentation

```
# previous simulation steps.
@in2 = 0.0
@in3 = 0.0
# ReadyToStart completed successfully
return true
end

# Statements entered in the trigger method are executed in each
# simulation step.
def trigger(t, te)
  # Compute output value (mean value of the current and
  # the previously saved input values).
  @output = (@input + @in2 + @in3) / 3.0

  # Shift on saved values
  @in3 = @in2
  @in2 = @input
end

end
```

The following configuration file shows how the Ruby script `RPUMeanValue.rb` can be integrated into the SILAB system.

```
SILAB Configuration
{
  Computerconfiguration Networks
  {
    Pool Workstation
    {
      Executeable = true;

      Computer LOCALHOST
      {
        IP = "127.0.0.1";
        PeriodMS = 100;
        Operator = true;
      };
    };
  };
  DPUConfiguration DPUs
  {
    Pool Test
    {
      Executeable = true;
    };
  };
}
```



SILAB DRIVING SIMULATION

version Version 7.1 documentation

```
DPUClock Clock
{
    Computer = {LOCALHOST};
    Index = 0;
};
DPUOscillator Speed
{
    Computer = {LOCALHOST};
    Start = 1;
    Period = 2000;
    On = 1;
    Index = 0;
    Amplitude = 125;
    Offset = 125;

};
DPURubyScript MeanValue
{
    Index = 1;
    Computer = {LOCALHOST};

    Script = "RPUMeanValue.rb";
    # Integrates the Ruby script.
    # It defines the inputs and
    # outputs of the DPURubyScript
    # instance MeanValue.

};
Connections =
{
    Speed.Out <-> MeanValue.Input
};
};
};
```